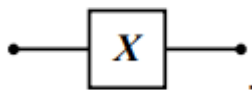


Рабочая тетрадь № 6

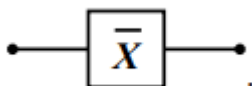
Релейно-контактная схема представляет собой схематическое изображение некоторого устройства, состоящего из переключателей, соединяющих их проводников, входов в схему и выходов из нее. В релейно-контактной схеме любой переключатель имеет только два состояния: замкнутое и разомкнутое.

1. Теоретический материал

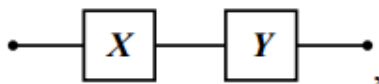
Простейшая схема состоит из одного переключателя X , одного входа и одного выхода. Переключателю X ставится в соответствие высказывание x – «Переключатель X замкнут». Если x истинно, то схема проводит электрический ток, в противном случае – не проводит. Если принимать во внимание только значение высказывания, то любому высказыванию x можно поставить в соответствие следующую двухполюсную релейно-контактную схему



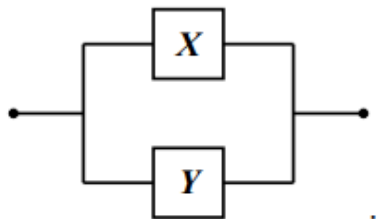
Отрицание высказывания x будем изображать двухполюсной схемой



Конъюнкцию двух высказываний x и y можно представить двухполюсной схемой с последовательным соединением переключателей X и Y



а дизъюнкцию – двухполюсной схемой с их параллельным соединением



Любая формула алгебры логики может быть представлена в нормальной

форме, следовательно, любой формуле алгебры логики можно поставить в соответствие некоторую релейно-контактную схему, и наоборот, каждой релейно-контактной схеме можно поставить в соответствие некоторую формулу алгебры логики.

2. Пример

Задача:

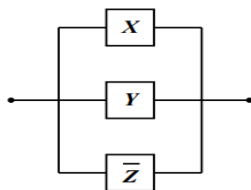
Дана логическая формула $(\bar{y} \wedge z) \rightarrow (x \vee y)$. Составьте для неё релейно-контактную схему.

Решение:

Упростим логическую формулу:

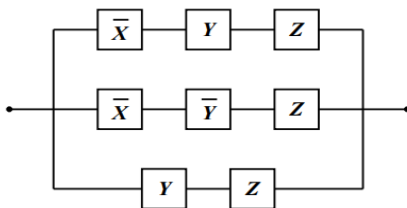
$$\begin{aligned} (\bar{y} \wedge z) \rightarrow (x \vee y) &= \text{тож. } 11 = \overline{\bar{y} \wedge z} \vee (x \vee y) = \\ &= \text{тож. } 5 = (\bar{\bar{y}} \vee \bar{z}) \vee (x \vee y) = \\ &= \text{тож. } 1, 3 = y \vee \bar{z} \vee x \vee y = \text{тож. } 8 = x \vee y \vee \bar{z}. \end{aligned}$$

Таким образом, релейно-контактная схема будет иметь следующий вид:



Задача:

Упростить релейно-контактную схему:



Решение:

Заменяем релейно-контактную схему на логическую формулу:

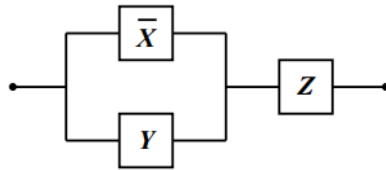
$$(\bar{x} \wedge y \wedge z) \vee (\bar{x} \wedge \bar{y} \wedge z) \vee (y \wedge z),$$

и упростим её:

$$\begin{aligned} (\bar{x} \wedge y \wedge z) \vee (\bar{x} \wedge \bar{y} \wedge z) \vee (y \wedge z) &= \text{тож. } 9 = (\bar{x} \wedge \bar{y} \wedge z) \vee (y \wedge z) = \\ &= \text{тож. } 4 = ((\bar{x} \wedge \bar{y}) \vee y) \wedge z = \text{тож. } 4 = ((\bar{x} \vee y) \wedge (\bar{y} \vee y)) \wedge z = \end{aligned}$$

$$= \text{тож. 7} = ((\bar{x} \vee y) \wedge 1) \wedge z = \text{тож. 6} = (\bar{x} \vee y) \wedge z.$$

Теперь заменим упрощённую логическую формулу обратно на релейно-контактную схему:



Задача:

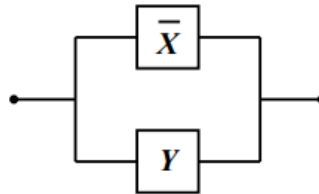
Составить релейно-контактную схему, соответствующую импликации.

Решение:

Выразим операцию импликации через основные логические операции «И», «ИЛИ» и «НЕ»:

$$x \rightarrow y = \text{тож. 11} = \bar{x} \vee y.$$

Построим по логической формуле контактно-релейную схему:



Задача:

Дана логическая формула:

$$((z \rightarrow x) \wedge (y \rightarrow z)) \rightarrow (y \rightarrow x).$$

Составьте для неё релейно-контактную схему.

Решение:

Упростим логическую формулу с помощью равносильных преобразований:

$$\begin{aligned} & ((z \rightarrow x) \wedge (y \rightarrow z)) \rightarrow (y \rightarrow x) = \\ & = \text{тож. 11} = ((\bar{z} \vee x) \wedge (\bar{y} \vee z)) \rightarrow (\bar{y} \vee x) = \\ & = \text{тож. 11} = \overline{(\bar{z} \vee x) \wedge (\bar{y} \vee z)} \vee (\bar{y} \vee x) = \\ & = \text{тож. 5, 3} = \bar{\bar{z} \vee x} \vee \bar{\bar{y} \vee z} \vee \bar{y} \vee x = \\ & = \text{тож. 5} = (\bar{\bar{z}} \wedge \bar{x}) \vee (\bar{\bar{y}} \wedge \bar{z}) \vee \bar{y} \vee x = \\ & = \text{тож. 1} = (z \wedge \bar{x}) \vee (y \wedge \bar{z}) \vee \bar{y} \vee x = \end{aligned}$$

$$\begin{aligned}
&= \text{тож. 3} = ((z \wedge \bar{x}) \vee x) \vee ((y \wedge \bar{z}) \vee \bar{y}) = \\
&= \text{тож. 4} = ((z \vee x) \wedge (\bar{x} \vee x)) \vee ((y \vee \bar{y}) \wedge (\bar{z} \vee \bar{y})) = \\
&= \text{тож. 7} = ((z \vee x) \wedge 1) \vee (1 \wedge (\bar{z} \vee \bar{y})) = \\
&= \text{тож. 6} = (z \vee x) \vee (\bar{z} \vee \bar{y}) = \text{тож. 7} = 1 \vee x \vee \bar{y} = \text{тож. 6} = 1
\end{aligned}$$

Составим по упрощённой логической формуле контактно-релейную схему:



Задача:

Построить релейно-контактную схему, соответствующую $f(x, y, z)$, если известно, что: $f(1, 0, 0) = f(0, 1, 1) = f(0, 1, 0) = f(0, 0, 0) = 1$.

Для остальных комбинаций аргументов значения функции равны нулю.

Решение:

Составим таблицу истинности для $f(x, y, z)$ и запишем по ней СДНФ.

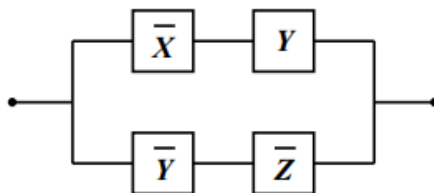
x	0	1	0	1	0	1	0	1
y	0	0	1	1	0	0	1	1
z	0	0	0	0	1	1	1	1
f	1	1	1	0	0	0	1	0

$$f_{\text{СДНФ}} = (\bar{x} \wedge \bar{y} \wedge \bar{z}) \vee (x \wedge \bar{y} \wedge \bar{z}) \vee (\bar{x} \wedge y \wedge \bar{z}) \vee (\bar{x} \wedge y \wedge z)$$

Упростим получившуюся СДНФ:

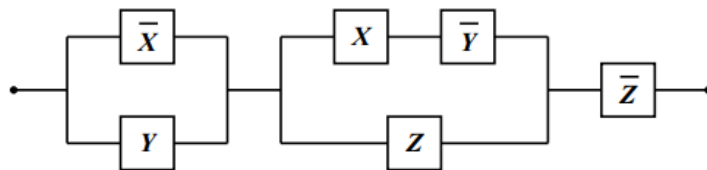
$$\begin{aligned}
f_{\text{СДНФ}} &= \text{тож. 4} = ((\bar{x} \vee x) \wedge (\bar{y} \wedge \bar{z})) \vee ((\bar{x} \wedge y) \wedge (\bar{z} \vee z)) = \\
&= \text{тож. 7} = (1 \wedge (\bar{y} \wedge \bar{z})) \vee ((\bar{x} \wedge y) \wedge 1) = \\
&= \text{тож. 6} = (\bar{y} \wedge \bar{z}) \vee (\bar{x} \wedge y).
\end{aligned}$$

Теперь построим контактно-релейную схему.



Задача:

Упростить релейно-контактную схему:

**Решение:**

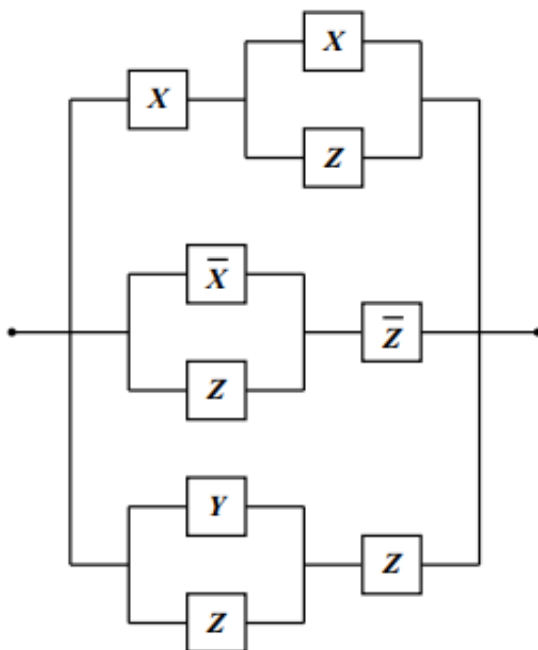
По контактно-релейной схеме запишем логическую формулу и далее упростим её:

$$\begin{aligned}
 & (\bar{x} \vee y) \wedge ((x \wedge \bar{y}) \vee z) \wedge \bar{z} = \\
 & = \text{тож. 5} = \overline{(x \wedge \bar{y})} \wedge ((x \wedge \bar{y}) \vee z) \wedge \bar{z} = \\
 & = \text{тож. 4} = \left(\left(\overline{(x \wedge \bar{y})} \wedge (x \wedge \bar{y}) \right) \vee \left(\overline{(x \wedge \bar{y})} \wedge z \right) \right) \wedge \bar{z} = \\
 & = \text{тож. 7} = \left(0 \vee \left(\overline{(x \wedge \bar{y})} \wedge z \right) \right) \wedge \bar{z} = \text{тож. 6,3} = \overline{(x \wedge \bar{y})} \wedge z \wedge \bar{z} = \\
 & = \text{тож. 7} = \overline{(x \wedge \bar{y})} \wedge 0 = \text{тож. 6} = 0.
 \end{aligned}$$

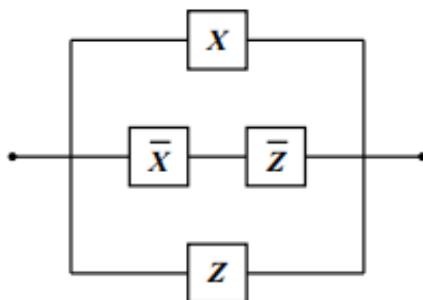
Составим по упрощённой логической формуле контактно-релейную схему.

**Задача:**

Доказать равносильность следующих релейно-контактных схем:



И

**Решение:**

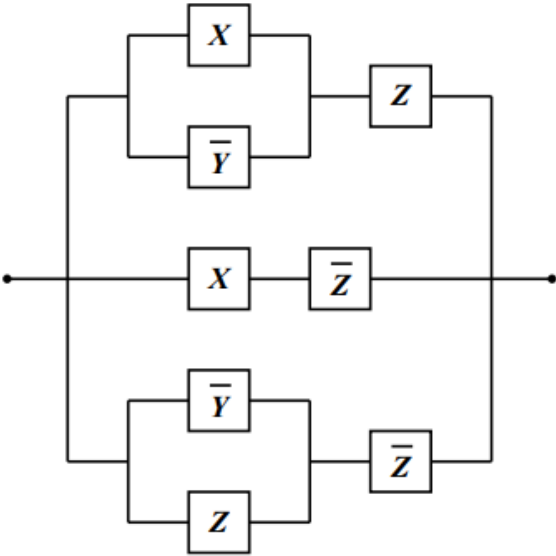
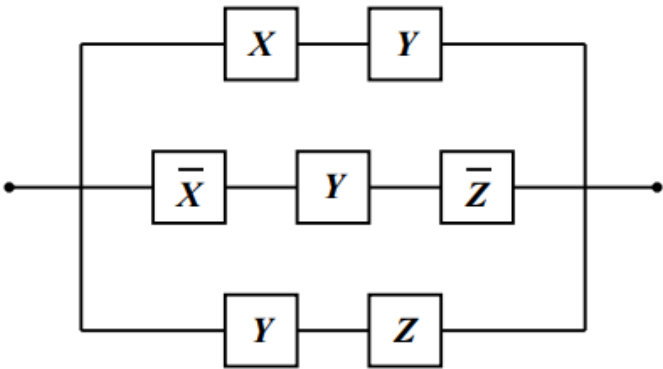
Запишем для каждой контактно-релейной схему логическую формулу и упростим их для дальнейшего сравнения.

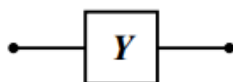
$$\begin{aligned}
 f_1 &= (x \wedge (x \vee z)) \vee ((\bar{x} \vee z) \wedge \bar{z}) \vee ((y \vee z) \wedge z) = \\
 &= \text{тож. 9,4} = x \vee ((\bar{x} \wedge \bar{z}) \vee (z \wedge \bar{z})) \vee z = \\
 &= \text{тож. 7} = x \vee ((\bar{x} \wedge \bar{z}) \vee 0) \vee z = \text{тож. 6} = x \vee (\bar{x} \wedge \bar{z}) \vee z \\
 f_2 &= x \vee (\bar{x} \wedge \bar{z}) \vee z
 \end{aligned}$$

Видно, что $f_1 = f_2$, а следовательно они эквивалентны. Отсюда контактно-релейные схемы также эквивалентны.

3. Задания

1.	Задача:
	Составить релейно-контактную схему, соответствующую эквивалентности.
	Решение:
2.	Задача:
	Составить релейно-контактные схемы, соответствующие следующим формулам:
	1) $(x \rightarrow \bar{x}) \rightarrow x$;
	2) $(x \rightarrow z) \wedge (z \rightarrow y)$.
	Решение:

3.	Задача: Построить релейно-контактные схемы для логических функций $f_i(x, y, z)$ если известно, что: 1) $f_1(1,1,0) = f_1(1,0,1) = 1$; 2) $f_2(1,1,1) = f_2(1,0,1) = f_2(0,1,0) = 1$; 3) остальные значения функций $f_i(x, y, z)$ равны нулю. Решение:
4.	Задача: Упростите релейно-контактную схему:  Решение:
5.	Задача: Докажите эквивалентность следующих релейно-контактных схем:  и



Решение:

1. Теоретический материал

Любое вычислительное устройство состоит из набора так называемых «вентилей» – минимальных логических устройств, способных преобразовывать входные сигналы в выходные. Ниже в таблице 1 представлены основные обозначения логических вентилях в различных стандартах.

Таблица 1 – Логические вентили и их условные графические обозначение в стандартах «ГОСТ 2.743-91», «IEC 60617-12: 1997» и «US ANSI 91-1984»

Название вентиля	Условные графические обозначения (УГО)		
	ГОСТ 2.743-91	IEC 60617-12: 1997	US ANSI 91-1984
НЕ			
И			
ИЛИ			
НЕ И			
НЕ ИЛИ			
исключающее ИЛИ			

Маленькие кружочки в УГО обозначают инверсию сигнала (т.е. отрицание НЕ). Они могут стоять и на входе вентиля, обозначая инверсию входного сигнала.

Рассмотрим первое простое устройство, которое можно собрать на вентилях: **полусумматор**. Его таблица истинности и схема представлена на рисунке 1.

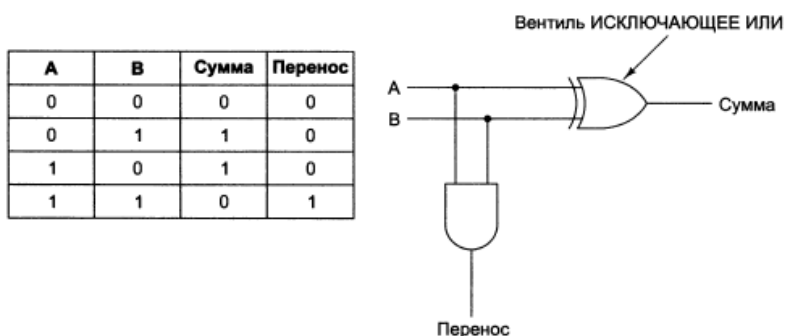
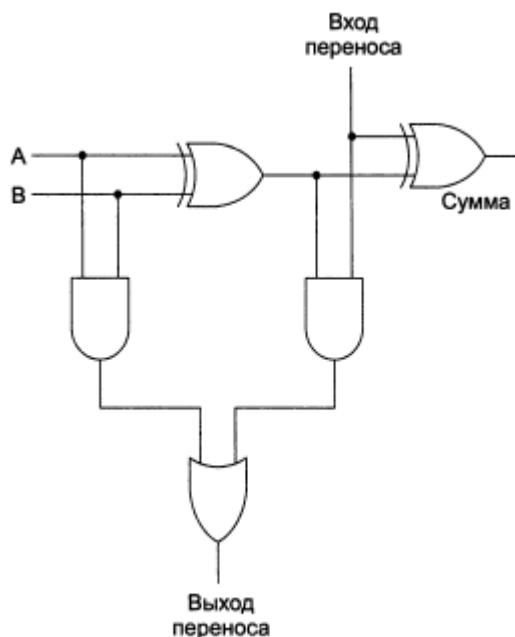


Рисунок 1 – Таблица истинности для сложения одноразрядных чисел и схема полусумматора

Полусумматор предназначен для нахождения суммы двух бит. При необходимости сложения чисел, состоящих из двух и более разрядов, эта схема уже не работает (нельзя просто взять необходимое число полусумматоров, т.к. в этой схеме не предусмотрен учёт переноса из младшего разряда в старший). Для этого нужно уже соединять между собой так называемые **полные сумматоры**. Таблица истинности и схема полного сумматора представлена на рисунке 2.

A	B	Вход переноса	Сумма	Выход переноса
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

а



б

Рисунок 2 – Таблица истинности для полного сумматора (а);
схема для полного сумматора (б)

В вычислительной технике также можно встретить применение **мультиплексоров**. Мультиплексор имеет один выход и две группы входных линий: информационные и адресные. Код, подаваемый на адресные линии, определяет, какой из информационных входов в данный момент подключен к выходному выводу. Поскольку n -разрядный двоичный код может принимать 2^n значений, то если число адресных входов мультиплексора равно n , число его информационных входов должно равняться 2^n . УГО мультиплексора с 8 информационными входными линиями $D_0, D_1, \dots, D_7$, 3 адресными входными линиями A, B, C и 1 выходной линией F приведено на рисунке 3.

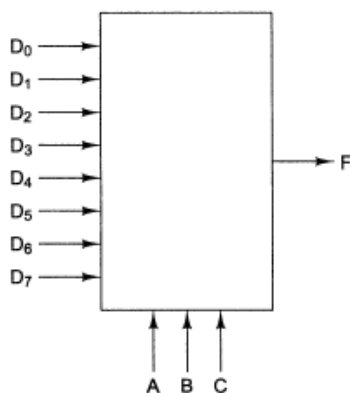


Рисунок 3 – Мультиплексор 8 в 1

Используя мультиплексор можно реализовывать логические функции (пример ниже).

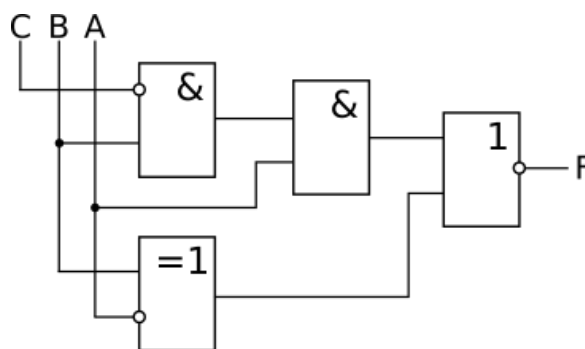
2. Пример

1. Задача:

Нарисовать схему (от руки или в любом графическом редакторе), реализующую следующую логическую функцию:

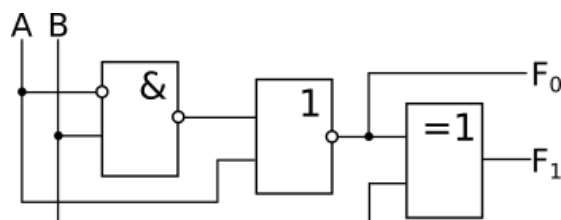
$$F = \overline{(A \wedge B \wedge \bar{C}) \vee (\bar{A} \oplus B)}.$$

Решение:



2. Задача:

Записать логическую формулу (формулы) и таблицу истинности для следующей схемы:



Решение:

$$F_0 = \overline{\overline{A} \wedge B} \vee A,$$

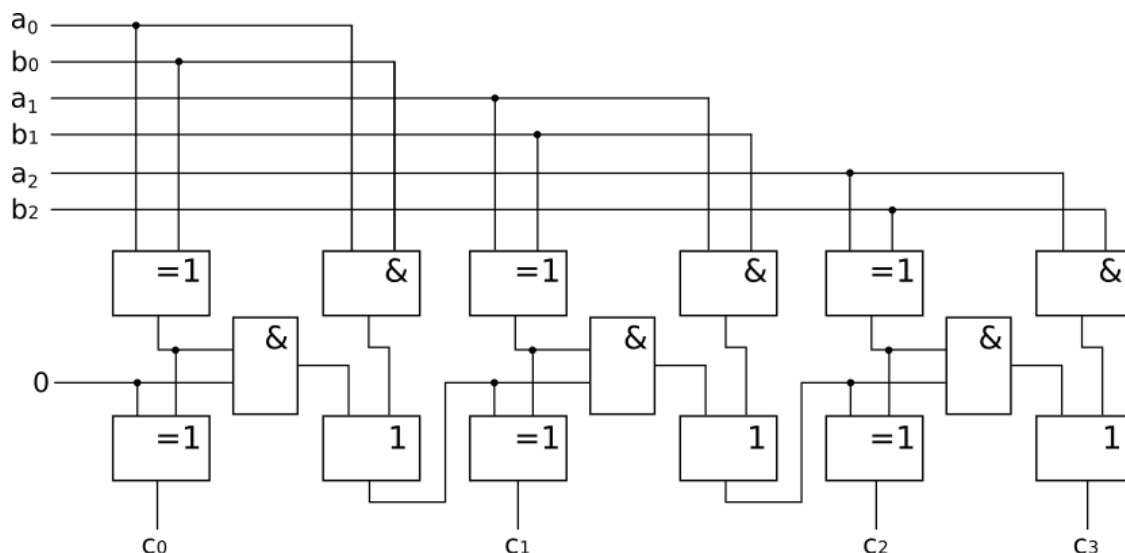
$$F_1 = F_0 \oplus B.$$

A	B	$\bar{A}$	$\bar{A} \wedge B$	$\overline{\bar{A} \wedge B}$	$\overline{\bar{A} \wedge B} \vee A$	F_0	F_1
0	0	1	0	1	1	0	0
0	1	1	1	0	0	1	0
1	0	0	0	1	1	0	0
1	1	0	0	1	1	0	1

3. Задача:

Нарисовать схему (от руки или в любом графическом редакторе), позволяющую суммировать трёхбитовые двоичные числа ($C = A + B$, $A = a_2a_1a_0$, $B = b_2b_1b_0$ и $C = c_3c_2c_1c_0$).

Решение:



4. Задача:

Используя мультиплексор, нарисовать схему (от руки или в любом графическом редакторе), реализующую функцию большинства от трёх логических переменных.

Решение:

Таблица истинности для функции большинства				Схема, реализующая необходимую логическую функцию	
A	B	C	F		
0	0	0	0		
0	0	1	0		
0	1	0	0		
0	1	1	1		
1	0	0	0		
1	0	1	1		
1	1	0	1		
1	1	1	1		

3. Задания

1. **Задача:**

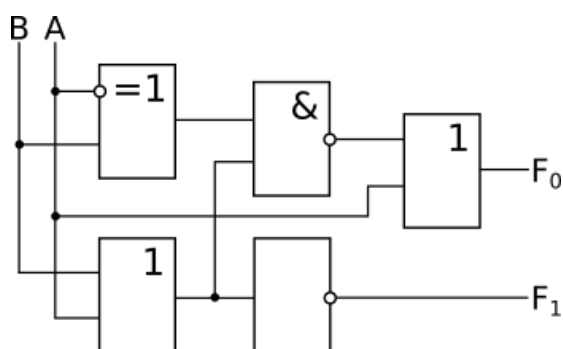
Нарисовать схему (от руки или в любом графическом редакторе), реализующую следующую логическую функцию:

$$(\overline{A \vee B \oplus C}) \wedge (\overline{BVC}) .$$

Решение:

2. **Задача:**

Записать логическую формулу (формулы) и таблицу истинности для следующей схемы:



Решение:

3. **Задача:**

Используя мультиплексор, нарисовать схему (от руки или в любом графическом редакторе), реализующую функцию импликации $\rightarrow$.

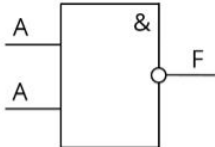
Решение:

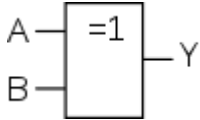
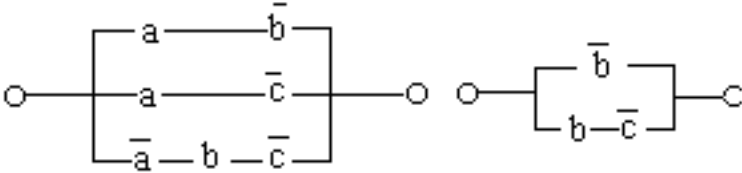
4. **Задача:**

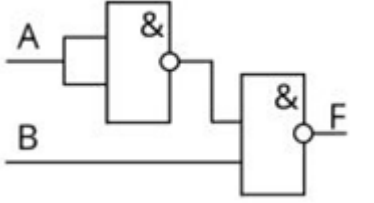
Нарисовать схему (от руки или в любом графическом редакторе), позволяющую умножать двухбитовые двоичные числа.

Решение:

Тест 6

1.	Для сложения многоразрядных двоичных чисел используется последовательность соединённых между собой:
	а) мультиплексоров; б) триггеров; в) полусумматоров; г) полных сумматоров.
	Ответ:
2.	Устройство, выполняющее базовые логические операции, называется:
	а) реестр; б) ячейка; в) вентиль; г) триггер
	Ответ:
3.	Графическое изображение логического выражения называется:
	а) схема б) рисунок в) чертеж г) график
	Ответ:
4.	Электронная схема, применяемая в регистрах компьютера для запоминания одного разряда двоичного кода это ...
	а) вентиль б) логическая схема в) триггер г) электронная схема
	Ответ:
5.	Мультиплексор – это устройство для ...
	а) умножения чисел б) коммутации одного из информационных входов с единственным выходом на основе кода, подаваемого на адресные входы в) произведения операции сдвига на основе данных на информационных входах и кода на адресных входах г) хранения одного бита информации
	Ответ:
6.	Какому логическому выражению соответствует схема на рисунке?
	

	a) $F = A$ в) $F = A \vee \bar{A}$ б) $F = A \wedge \bar{A}$ г) $F = \bar{A}$
	Ответ:
7.	Какой минимально необходимо взять мультиплексор, чтобы реализовать заданную таблицей истинности логическую функцию от 4 логических переменных? а) с 4 информационными входными линиями, 2 адресными входными линиями и 1 выходной линией б) с 16 информационными входными линиями, 4 адресными входными линиями и 1 выходной линией в) с 4 информационными входными линиями, 4 адресными входными линиями и 2 выходными линиями г) с 4 информационными входными линиями, 16 адресными входными линиями и 2 выходными линиями Ответ:
8.	Такой структурной схемой обозначается  а) исключающее ИЛИ в) бинарное НЕ б) дизъюнктор г) инвертор Ответ:
9.	Равносильны или нет следующие релейно-контактные схемы?  Ответ:

10.	Какому логическому выражению соответствует схема на рисунке?
	
	а) $F = \overline{A} \wedge \overline{B}$ б) $F = \overline{A} \wedge \overline{\overline{B}}$
	в) $F = \overline{\overline{A}} \wedge \overline{B}$ г) $F = \overline{\overline{A}} \wedge B$
	Ответ:

Реализация задач на языке программирования Python

В языке Python имеются множество удобных контейнеров для хранения различной информации: строки, списки, кортежи, множества и словари. Их использование обеспечивает эффективность решения широкого спектра задач.

1. Теоретический материал

Строки - это последовательности символов. Создать строки можно следующим образом:

```
s = "Hello, World!"
p = 'Миру - мир!'
```

Доступ к символу с индексом n (начиная с нуля) осуществляется с помощью выражения s[n]. Подстроку можно выделить, указав интервал s[n:m]. Такая возможность в языке называется срезом. Пример:

```
s = "Мамамылараму"
print(s[6])
p = s[5:9]
print(p)
```

Если индекс отрицательный, то отсчет идет от конца строки. При этом индекс -1 соответствует последнему символу. Строки в Python не изменяемы, т.е. по индексу можно только получить символ, но не поменять его!

Поскольку строка задается с помощью кавычек (двойных или

одинарных), то для того, чтобы в самой строке использовать символы кавычек, их необходимо экранировать следующим образом:

```
s = "А Балда приговаривал с укоризной: \"Не гонялся бы ты, поп, за  
дешевизной\""  
print(s)
```

Также можно воспользоваться одинарными кавычками, чтобы можно было использовать двойные (и наоборот):

```
s = 'А Балда приговаривал с укоризной: "Не гонялся бы ты, поп, за  
дешевизной"'  
print(s)
```

В строках можно использовать специальные символы (управляющие последовательности), которые задаются следующим образом:

\n – новая строка;

\t – табуляция.

Пример красивого вывода:

```
s = "\n\n\t\tА Балда приговаривал с укоризной:\n \n\t\t\"Не гонялся бы ты, поп, за дешевизной\""\n\n"  
print(s)
```

Со строками можно выполнять операции конкатенации с помощью оператора "+", который мы уже проходили. Строку можно умножить на целое число **n**, в результате чего строка будет повторена **n** раз:

```
s = "Мама"  
s = s * 3  
print(s)
```

Узнать длину строки можно с помощью функции **len(s)**.

Часто возникает задача поиска подстроки в строке. Для этих целей используется функция **s.find(str)**:

```
s = "Жили у Бабуся два веселых гуся"  
n = s.find("два")  
print(n)
```

С помощью функции **s.split(str)** строки можно разделять по заданному

СИМВОЛУ:

```
s = "Мама | Папа | Я"
```

```
S = s.split('|')
```

```
print(S[1])
```

Полный список функций для строк:

- <https://pythonworld.ru/typy-dannyx-v-python/stroki-funkcii-i-metody-strok.html>

2. Пример

Задача:

Удалить все пробелы из данной строки.

Решение (код программы):

```
txt = "I like bananas"
```

```
x = txt.replace(" ", "")
```

```
print(x)
```

3. Задания

1.

Задача:

Пользователь вводит произвольное слово. Необходимо проверить, что оно является палиндромом. **Палиндромом** называется слово, которое одинаково читается как слева направо, так и справа налево, например, в английском языке такими словами являются "radar" и "racecar".

Решение (код программы):

2.

Задача:

Пользователь вводит строку. Необходимо вывести её на экран в нижнем регистре.

Решение (код программы):

3.	Задача:
	 Пользователь вводит строку. Необходимо вывести её на экран в верхнем регистре.
	Решение (код программы):
	
4.	Задача:
	 Пользователь вводит строку. Необходимо проверить есть ли в ней числа (цифры).
	Решение (код программы):
	
5.	Задача:
	 Задана строка. Необходимо между её символами вставить по одному пробелу.
	Решение (код программы):
	
6.	Задача:
	 Дан символ С. Вывести его код (то есть номер в кодовой таблице).
	Решение (код программы):
	

1. Теоретический материал

В Python списки используются для хранения данных и позволяют получать доступ к ним по индексу (их порядковому номеру). Другими словами списки в Python реализуют такую структуру данных как массивы, но добавляют и некоторые специфические для себя операции.

A = [] # пустой список

B = [1, 1, 2, 3, 5, 8, 13, 21] # числа Фибоначчи

C = ["мама", 120, 'z', "список"] # смешанный список

D = [[1, 2], ['a', 'b', 'c'], [[3.14, "abc"], [120, 140]]] # список списков

К элементам списка можно обращаться через индексы, которые начинаются с нуля. В отличие от строк, элементы списков можно заменять:

```
A = [1, 2, 3]
A[1] = "новое значение"
print(A)
```

Для работы со списками существует большое количество методов. Вот основные:

Метод	Описание
list.append(x)	добавляет элемент в конец списка
list.insert(i, x)	вставляет на i-ый элемент значение x
list.count(x)	возвращает количество элементов со значением x
list.reverse()	разворачивает список
list.clear()	очищает список

В Python есть неизменяемый аналог рассмотренного выше списка: кортежи. За счёт неизменяемости интерпретатор Python работает с кортежами эффективнее. Создать кортеж можно следующим образом:

```
A = ("Ivanov", "Ivan", 1988)
Surname, Name, Date = A
print(Surname + " " + Name + " " + Date.__str__())
```

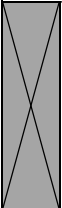
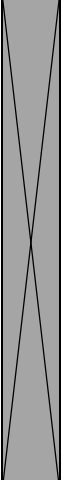
2. Пример

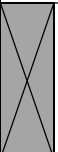
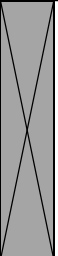
Задача:

Заполните список 20 случайными числами.

Решение (код программы):

```
import random
a = []
for i in range(1,21):
    a.append(random.randint(5,10))
print(a)
```

Задача:	
	Напишите программу, которая запрашивает с ввода восемь чисел, добавляет их в список. На экран выводит их сумму, максимальное и минимальное из них. Для нахождения суммы, максимума и минимума воспользуйтесь встроенными в Python функциями <code>sum()</code> , <code>max()</code> и <code>min()</code> .
Решение (код программы):	
	<pre> l = [] for i in range(0,8): a = int(input()) l.append(a) print(l) print(sum(l)) print(min(l)) print(max(l)) </pre>

3. Задания	
1.	Задача:  Если объект <code>range</code> (диапазон) передать встроенной в Python функции <code>list()</code> , то она преобразует его к списку. Создайте, таким образом, список с элементами от 0 до 100 и шагом 17. Решение (код программы): 
2.	Задача:  Напишите программу, которая заполняет список пятью словами, введенными с клавиатуры, измеряет длину каждого слова и добавляет полученное значение в другой список. Например, список слов – ['yes', 'no', 'maybe', 'ok', 'what'], список длин – [3, 2, 5, 2, 4]. Оба списка должны выводиться на экран. Решение (код программы): 

1. Теоретический материал

Теперь рассмотрим множества в Python. Множества отличаются от списков и кортежей тем, что хранящиеся в них объекты являются неупорядоченными. Кроме того, множества содержат только уникальные элементы. Зато с множествами можно производить стандартные операции объединения, пересечения и разности

Кроме того, множества содержат только уникальные элементы. Зато с множествами можно производить стандартные операции объединения, пересечения и разности:

Операция	Описание операции
	объединение
&	пересечение
-	разность
^	симметричная разность

Множество можно создать с помощью оператора **set()**:

```
A = set(['a', 'apple', 120])
```

Добавить один элемент в множество можно с помощью метода **A.add(x)**. Чтобы добавить другое множество используется операция **A.update(container)**. Для удаления элементов используется метод **A.remove(x)**. Примеры работы с множествами:

```
A = set(["apple", "orange"])
A.add("pear")
B = set(["apple", "pear", "banana"])
C = A | B
D = A&B
print(C)
print(D)
```

В Python есть контейнер, который называется словарем. Словари позволяют хранить связки «ключ–значение» и осуществлять доступ к элементам по ключу. Пример:

```
people = {
    "Name": "Ivan",
    "Surname": "Ivanov",
```

```

    "Date": 1988
}
print(people)
people["Name"] = "Alex"
people["Date"] = 2008
print(people)

```

2. Задания

1.	Задача:
	Даны два множества X и Y. Найти: 1) их объединение 2) их пересечение 3) элементы множества A, которых нет во множестве B
	Решение (код программы):